MANDALA: An E(3)-Equivariant Graph Neural Network Framework for Learning Sparse Electronic-Structure Hamiltonian and Density Matrices with Operator-Derived Observable Guidance

Bartosz Brzoza[1,2,3], Wiktoria Szopa[1,2,4], Zakaria Elabid[1,2], Vincent Martinetto[1,2], Mani Lokamani[2], Thomas D. Kühne[1,2], Attila Cangi[1,2,*]

[1] *Center for Advanced Systems Understanding, 02826 Görlitz, Germany*
[2] *Helmholtz-Zentrum Dresden-Rossendorf, 01328 Dresden, Germany*
[3] *Institute of Computer Science, University of Wrocław, 50-300 Wrocław, Poland*
[4] *Faculty of Physics, Warsaw University of Technology, 00-662 Warsaw, Poland*

Abstract

Electronic-structure calculations based on Kohn-Sham density functional theory remain indispensable in computational materials science and chemistry. Their computational cost, however, limits accessible system sizes and simulation times. It also makes systematic exploration of model architectures and training strategies on top of such data expensive. At the same time, conventional machine-learning interatomic models usually target only energies and forces, which leaves out the operator-level electronic information required for many scientifically relevant analyses. `Mandala` was developed to address this gap by providing a framework for learning sparse electronic-structure operators directly, while preserving access to derived observables and Hamiltonian-based analysis. We present `Mandala`, a modular software framework for learning block-sparse electronic-structure matrices with E(3)-equivariant graph neural networks. The framework is built around a unified representation of atom-resolved Hamiltonian, overlap, and density matrices, together with reusable abstractions for basis conversion, sparse block handling, irrep mapping, graph construction, model definition, and training. This design allows `Mandala` to support multiple electronic-structure

*Corresponding author
Email address: `a.cangi@hzdr.de` (Attila Cangi)

backends, heterogeneous chemical compositions, and a wide range of neural architecture variants within one workflow.

A distinguishing feature of `Mandala` is that it does not stop at matrix reconstruction. Predicted operators can be contracted to obtain the band-energy observable and electron count, while the predicted Hamiltonian preserves access to electronic post-processing, including band-structure analysis. This allows the software to bridge electronic-structure learning and observable-guided modeling while retaining a representation tied to quantum-mechanical operators rather than only scalar or vector targets.

`Mandala` combines a modular software architecture with an operator-centered learning formulation for sparse electronic-structure prediction. The present paper emphasizes the block sparse matrix data model, the observable-guidance loss design, and the broad hyperparameter and architecture search space exposed by the framework. In this form, `Mandala` is intended to support machine-learning workflows that go beyond conventional atomistic interatomic potentials by resolving electronic structure and operator-derived observables within one scalable implementation.

Keywords: Electronic structure, Density functional theory, Equivariant graph neural networks, Sparse matrix learning, Operator-derived observables, Scientific machine learning

---

PROGRAM SUMMARY

Program Title: `Mandala`
CPC Library link to program files: [TODO: to be added by the Technical Editor]
Developer's repository link: `https://github.com/Mandala-org/Mandala`
Licensing provisions: Apache-2.0
Programming language: Python
Nature of problem: Electronic-structure calculations based on Kohn-Sham density functional theory are the central computational method in computational materials science and chemistry, but their computational cost limits accessible system sizes, trajectory lengths, and large-scale model-selection studies. Atomistic simulations based on machine-learned interatomic potentials alleviate this cost for energies and forces, yet they generally do not expose the electronic operators needed for richer observables or electronic post-processing and cannot be used to model processes that involve electronic charge transfer reliably. A complementary route is to learn sparse representations of electronic-structure matrices themselves. Such a computational method must support multiple atom types, changing orbital bases,

periodic boundary conditions, equivariant geometric processing, and physically meaningful operator-derived observables.
Solution method: `Mandala` represents Hamiltonian, overlap, and density matrices as atom-pair-resolved sparse block tensors and maps these blocks to irreducible E(3) representations compatible with equivariant neural networks. The package combines parser modules for electronic-structure data, basis-conversion utilities, a shared irrep mapper, graph construction routines, and configurable E(3)-equivariant message-passing models. Multi-head prediction allows simultaneous learning of several matrices, while band-energy and electron-count observables are evaluated from the predicted operators using differentiable sparse traces and may be included as training objectives. The framework is organized to support extensibility, architecture search, and reproducible training workflows.
Additional comments including restrictions and unusual features: The framework is optimized for sparse matrix learning of electronic structure in atomistic systems with localized orbital bases and periodic or non-periodic geometries. A distinctive feature of `Mandala` is the combination of operator-level learning with optional observable guidance derived from those operators, which allows the same software stack to serve both electronic-structure emulation and observable-guided atomistic applications. The package is designed to accommodate multiple data backends together with a broad search space of equivariant architectures and training objectives.

1. Introduction

Kohn-Sham density functional theory (DFT) is the standard method of electronic-structure simulation because it offers a practical compromise between physical fidelity and computational cost across chemistry, condensed-matter physics, and materials science [1, 2]. Even so, both cubic scaling with system size and repeated self-consistent solution of the Kohn-Sham equations remains a major bottleneck in workflows that require large supercells, long molecular-dynamics trajectories, broad compositional screening, or repeated re-evaluation under changing thermodynamic conditions. This cost has motivated sustained interest in machine-learned surrogates for atomistic and quantum-mechanical simulation. Relevant directions include learned potential-energy surfaces and interatomic forces, as well as learned electron densities, wave-function-related quantities, kinetic-energy functionals, and reduced electronic representations [3–12].

Many successful atomistic machine-learning approaches bypass the explicit electronic structure and instead learn scalar or vector observables such as energies and forces. This strategy is highly effective for interatomic potentials, from early neural-network potential formulations to modern equivariant graph models, and recent work has shown that equivariant neural architectures are especially well suited for energy and force prediction because they can encode rotational structure directly in the model rather than relying entirely on handcrafted invariants or data augmentation [13, 14]. However, learning only energies and forces also limits direct access to the electronic structure itself, including operator-valued quantities and Hamiltonian-derived analyses such as band structure. For applications that remain fundamentally electronic-structure-centered, it is attractive to learn the operators themselves. Examples include systems in which charge transfer, orbital hybridization, defect-localized states, response to external fields, or band-structure-level analysis are central to the scientific question. In such settings, access to the underlying electronic operators is not merely auxiliary metadata but part of the modeling target itself.

`Mandala` is designed around this operator-learning viewpoint. The framework targets block-sparse matrix quantities arising in localized-orbital electronic-structure calculations, in particular the Hamiltonian, overlap, and density matrices. Instead of treating these objects as dense global arrays, `Mandala` represents them as atom-pair-resolved sparse blocks and combines this representation with E(3)-equivariant graph neural networks. The resulting learning problem respects both the locality of the underlying electronic structure and the geometric transformation laws imposed by three-dimensional space.

The development of `Mandala` is motivated by three specific design goals. First, it should be modular at the level of both scientific abstractions and code organization. Data ingestion, basis conversion, graph construction, irrep mapping, model definition, and training logic should be independently replaceable while still composing into a single workflow. Second, the framework should support supervision beyond raw matrix reconstruction. When Hamiltonian, overlap, and density matrices are predicted jointly, the band-energy observable and electron count can be computed directly from the predicted operators and included in the training objective. Third, the framework should make architecture exploration an inherent capability rather than an afterthought. Choices such as irreducible feature content, message-passing depth, tensor-product design, head structure, residual connections, normalization, and observable losses must all be accessible through a coherent con-

figuration and sweep interface.

Related operator-learning efforts include the DeepH family of Hamiltonian-learning models for localized-basis DFT, including their original message-passing formulation, extensions to magnetic superstructures, an explicitly E(3)-equivariant formulation, hybrid-functional Hamiltonians, and conversion pathways from plane-wave data to localized-orbital Hamiltonians [15–20]. These works demonstrate the promise of Hamiltonian learning at scale and provide a close point of comparison for the operator-centric aims of `Mandala`, while our emphasis here is on a reusable software framework that unifies sparse operator prediction, observable guidance, multi-backend data handling, and broad architecture exploration.

These goals distinguish `Mandala` from earlier machine-learning workflows in two ways. Relative to purely observable-driven interatomic potential models (MLIPs), `Mandala` preserves an operator-level representation that remains useful for electronic-structure analysis. Relative to monolithic research scripts built around one specific dataset or one fixed neural architecture, it provides a reusable framework in which data formats, physical targets, and model variants can be exchanged without rewriting the entire code path. In this sense, `Mandala` is intended as a unified software environment for sparse operator learning, observable guidance, and architecture exploration.

The following sections describe the sparse matrix data model, the E(3)-equivariant learning formulation, the observable-aware loss construction, and the software abstractions that make these components composable.

2. Theoretical Background

2.1. From the full electron-nuclear Schrödinger equation to Kohn-Sham DFT

At the most fundamental nonrelativistic level, an atomistic system is described by the stationary many-body Schrödinger equation for all electrons and all nuclei,

$$\hat{H}(\underline{\boldsymbol{r}};\underline{\boldsymbol{R}})\Psi(\underline{\boldsymbol{r}};\underline{\boldsymbol{R}}) = E\Psi(\underline{\boldsymbol{r}};\underline{\boldsymbol{R}}), \tag{1}$$

where $\Psi(\underline{\boldsymbol{r}};\underline{\boldsymbol{R}})$ denotes the many-body wavefunction, $\underline{\boldsymbol{r}}$ the collection of electronic coordinates, and $\underline{\boldsymbol{R}}$ the collection of ionic coordinates. The semicolon indicates that the dependence on ionic coordinates is parametric within the Born-Oppenheimer approximation. We adopt atomic units throughout, with $\hbar = m_e = e^2 = 1$, so that energies are measured in Hartree and lengths in Bohr radii.

Under the Born-Oppenheimer approximation [21], the electronic Hamiltonian is written as

$$\hat{H}(\underline{\boldsymbol{r}};\underline{\boldsymbol{R}}) = \hat{T}_e(\underline{\boldsymbol{r}}) + \hat{V}_{ee}(\underline{\boldsymbol{r}}) + \hat{V}_{ei}(\underline{\boldsymbol{r}};\underline{\boldsymbol{R}}) + E_{ii}(\underline{\boldsymbol{R}}), \tag{2}$$

with

$$\hat{T}_e(\underline{\boldsymbol{r}}) = -\frac{1}{2}\sum_{j=1}^{N_e}\nabla_j^2, \tag{3}$$

$$\hat{V}_{ee}(\underline{\boldsymbol{r}}) = \frac{1}{2}\sum_{j=1}^{N_e}\sum_{k\neq j}^{N_e}\frac{1}{|\boldsymbol{r}_j - \boldsymbol{r}_k|}, \tag{4}$$

$$\hat{V}_{ei}(\underline{\boldsymbol{r}};\underline{\boldsymbol{R}}) = -\sum_{j=1}^{N_e}\sum_{\alpha=1}^{N_i}\frac{Z_\alpha}{|\boldsymbol{r}_j - \boldsymbol{R}_\alpha|}, \tag{5}$$

and

$$E_{ii}(\underline{\boldsymbol{R}}) = \frac{1}{2}\sum_{\alpha=1}^{N_i}\sum_{\beta\neq\alpha}^{N_i}\frac{Z_\alpha Z_\beta}{|\boldsymbol{R}_\alpha - \boldsymbol{R}_\beta|}. \tag{6}$$

The ion-ion term amounts to a constant energy shift for fixed ionic configuration, while the Hamiltonian and wavefunction depend only parametrically on $\boldsymbol{R}$. In the following, this parametric dependence is not always written explicitly.

The central remaining difficulty is therefore the interacting many-electron problem defined by $\hat{T}_e + \hat{V}_{ee} + \hat{V}_{ei}$ at fixed ionic geometry.

Density functional theory (DFT) replaces the many-electron wavefunction by the electron density $n(\boldsymbol{r})$ as the central variable [1]. In the Kohn-Sham construction [2], the interacting electron problem is mapped onto a fictitious system of non-interacting electrons governed by

$$\left[-\frac{1}{2}\nabla^2 + v_S(\boldsymbol{r})\right]\psi_j(\boldsymbol{r}) = \epsilon_j\psi_j(\boldsymbol{r}), \tag{7}$$

where $j = 1,\ldots,N_e$ labels the Kohn-Sham orbitals. The corresponding electronic density is

$$n(\boldsymbol{r}) = \sum_{j=1}^{N_e}|\psi_j(\boldsymbol{r})|^2. \tag{8}$$

The Kohn-Sham potential is

$$v_S(\boldsymbol{r}) = \frac{\delta E_H[n]}{\delta n(\boldsymbol{r})} + \frac{\delta E_{XC}[n]}{\delta n(\boldsymbol{r})} + v_{ei}(\boldsymbol{r}), \tag{9}$$

where $E_H[n]$ is the Hartree energy, $E_{XC}[n]$ the exchange-correlation energy, and

$$v_{ei}(\boldsymbol{r}) = -\sum_{\alpha} \frac{Z_\alpha}{|\boldsymbol{r} - \boldsymbol{R}_\alpha|}. \tag{10}$$

Although the Kohn-Sham construction is formally exact, practical calculations rely on approximations to E_{XC}.

In this notation, the ground-state energy functional is written as

$$E[n] = T_S[n] + E_H[n] + E_{XC}[n] + \int d\boldsymbol{r}\, n(\boldsymbol{r}) v_{ei}(\boldsymbol{r}) + E_{ii}, \tag{11}$$

where $T_S[n]$ is the Kohn-Sham kinetic energy. This is the DFT-level starting point that `Mandala` connects to localized-orbital operator learning.

`Mandala` operates on localized-orbital matrix representations of this problem. Expanding the Kohn-Sham orbitals in a finite basis $\{\chi_\mu\}$,

$$\psi_j(\boldsymbol{r}) = \sum_{\mu} C_{\mu j} \chi_\mu(\boldsymbol{r}), \tag{12}$$

leads to the generalized eigenvalue problem

$$\sum_{\nu} H_{\mu\nu} C_{\nu j} = \epsilon_j \sum_{\nu} S_{\mu\nu} C_{\nu j}, \tag{13}$$

with

$$H_{\mu\nu} = \langle \chi_\mu | \hat{H}_{\mathrm{KS}} | \chi_\nu \rangle, \qquad S_{\mu\nu} = \langle \chi_\mu | \chi_\nu \rangle. \tag{14}$$

The one-particle density matrix is

$$D_{\mu\nu} = \sum_{j} f_j C_{\mu j} C^*_{\nu j}, \tag{15}$$

where f_j denotes the orbital occupation. In a nonorthogonal basis, the electron count is

$$N_e = \mathrm{Tr}(DS), \tag{16}$$

while the band energy observable is obtained from

$$E_{\mathrm{b}} = \mathrm{Tr}(DH). \tag{17}$$

In this work, `Mandala` uses the trace expressions above as operator-derived observables associated with the predicted matrices.

Localized basis sets also induce block sparsity. If orbital μ belongs to atom i and orbital ν belongs to atom j, then matrix entries can be organized into atom-pair blocks $X_{ij}^{\boldsymbol{L}}$, where $X \in \{H, S, D\}$ and $\boldsymbol{L} \in \mathbb{Z}^3$ indexes periodic lattice images. This sparse block structure is the operator domain learned by `Mandala`.

2.1.1. Hamiltonian gauge invariance and the overlap-shift correction

In a nonorthogonal basis, the generalized eigenproblem is invariant under a rigid shift of the Hamiltonian by a multiple of the overlap matrix. Let

$$HC = SC\varepsilon, \tag{18}$$

and define a shifted Hamiltonian

$$\widetilde{H} = H - \alpha S. \tag{19}$$

Then

$$\widetilde{H}C = (H - \alpha S)C = SC\varepsilon - \alpha SC = SC(\varepsilon - \alpha I). \tag{20}$$

Therefore the generalized eigenvectors are unchanged, while all eigenvalues are shifted by the same constant α. This proves that the choice of zero of energy can be represented, in matrix form, as the transformation $H \mapsto H - \alpha S$. For a fixed electron number and a geometry-independent shift α, one also finds

$$\mathrm{Tr}(D\widetilde{H}) = \mathrm{Tr}(DH) - \alpha \mathrm{Tr}(DS) = \mathrm{Tr}(DH) - \alpha N_e, \tag{21}$$

so only the absolute energy reference changes.

`Mandala` exploits this gauge freedom through the overlap-shift correction μ_H, which is chosen to minimize the mean-squared discrepancy between a predicted Hamiltonian and a reference Hamiltonian after subtraction of a multiple of the overlap matrix:

$$\mu_H = \arg\min_{\mu} \left\| H^{\mathrm{pred}} - H^{\mathrm{ref}} - \mu S \right\|_F^2 . \tag{22}$$

Taking the derivative with respect to μ and setting it to zero yields

$$\mu_H = \frac{\left\langle H^{\mathrm{pred}} - H^{\mathrm{ref}}, S \right\rangle_F}{\langle S, S \rangle_F}, \tag{23}$$

where $\langle A, B \rangle_F = \sum_{\mu\nu} A_{\mu\nu} B_{\mu\nu}$ is the Frobenius inner product, evaluated in `Mandala` blockwise over the aligned sparse representation. The corrected Hamiltonian $H^{\mathrm{pred}} - \mu_H S$ therefore removes a pure gauge offset without altering the physically relevant eigenvectors.

2.2. E(3) symmetry, equivariance, and irreducible representations

Atomistic systems in three-dimensional space are naturally acted on by the Euclidean group $\mathrm{E}(3) = \mathbb{R}^3 \rtimes \mathrm{O}(3)$, combining translations, rotations, and reflections. For a group element $g = (Q, \boldsymbol{t})$, with $Q \in \mathrm{O}(3)$ and $\boldsymbol{t} \in \mathbb{R}^3$, atomic positions transform as

$$\boldsymbol{R}_i \mapsto g \cdot \boldsymbol{R}_i = Q\boldsymbol{R}_i + \boldsymbol{t}. \tag{24}$$

A model output $f(X)$ is invariant if $f(g \cdot X) = f(X)$, and equivariant if

$$f(g \cdot X) = \rho_{\mathrm{out}}(g)\, f(X), \tag{25}$$

for a representation ρ_{out} appropriate to the predicted object. Scalar observables such as total energies are invariant. Vectors such as forces transform covariantly, and operator-valued quantities inherit structured transformation rules from the orbital basis in which they are expressed.

In equivariant neural networks, hidden features are not treated as generic channels. Instead, they are decomposed into irreducible representations (irreps) of O(3), labeled by angular momentum ℓ and parity $p \in \{+1, -1\}$. A feature space therefore takes the form

$$\mathcal{H} = \bigoplus_{\ell,p} m_{\ell,p}\, D^{(\ell,p)}, \tag{26}$$

where $m_{\ell,p}$ is the multiplicity of irrep (ℓ, p) and $D^{(\ell,p)}$ is the corresponding representation matrix. Scalars correspond to $(\ell = 0, p = +1)$, ordinary vectors to $(\ell = 1, p = -1)$, and higher-rank objects are built from larger ℓ channels. This representation-theoretic bookkeeping is particularly natural for localized atomic orbitals, whose angular content is already organized by orbital angular momentum.

The key architectural consequence is that every learned operation must respect the decomposition into irreps. Node and edge features are updated through maps that commute with the group action, rather than by unrestricted dense linear layers. Geometric information enters through relative displacement vectors and their spherical-harmonic expansions, while translational invariance is enforced by expressing all geometry in relative coordinates. Reflections are tracked through the parity label p, while rotations act within each ℓ channel. In this way, `Mandala` preserves directional information throughout the network without sacrificing the required covariance laws [13, 14].

2.3. Tensor products, spherical harmonics, and the Wigner-Eckart viewpoint

The basic algebraic operation of an equivariant graph neural network is the tensor product of irreps. If two feature channels transform according to $D^{(\ell_1,p_1)}$ and $D^{(\ell_2,p_2)}$, then their tensor product decomposes as

$$D^{(\ell_1,p_1)} \otimes D^{(\ell_2,p_2)} = \bigoplus_{L=|\ell_1-\ell_2|}^{\ell_1+\ell_2} D^{(L,p_1p_2)}. \tag{27}$$

This Clebsch-Gordan structure determines which output irreps may be produced by combining two input channels. In practical terms, it is the algebraic rule that governs message passing, nonlinear mixing of hidden features, and the final projection from latent features to matrix coefficients.

Edge geometry is injected through spherical harmonics $Y_{\ell m}(\hat{\boldsymbol{r}}_{ij})$, evaluated on normalized relative displacement directions. Because spherical harmonics transform irreducibly under rotations, they provide the canonical angular basis for equivariant edge messages. A typical equivariant interaction therefore combines learned node or edge features with spherical-harmonic features through tensor products, while radial functions modulate the interaction strength as a function of distance.

The same representation-theoretic structure appears in matrix elements of operators between angular-momentum-carrying orbitals. For a spherical tensor operator $T_q^{(k)}$, the Wigner-Eckart theorem states that

$$\langle \alpha \ell m | T_q^{(k)} | \alpha' \ell' m' \rangle = \langle \ell' m'; kq | \ell m \rangle \langle \alpha \ell \| T^{(k)} \| \alpha' \ell' \rangle, \tag{28}$$

where ℓ and ℓ' denote orbital angular momenta, m and m' their magnetic quantum numbers, $T_q^{(k)}$ is the qth component of a spherical tensor operator of

rank k, α and α' collect any additional quantum numbers needed to specify the states, and $\langle \ell' m'; kq | \ell m \rangle$ is a Clebsch-Gordan coefficient. The double-bar matrix element $\langle \alpha \ell \| T^{(k)} \| \alpha' \ell' \rangle$ is the corresponding reduced matrix element. that is, the dependence on magnetic quantum numbers is separated from the reduced matrix element. `Mandala` uses this viewpoint operationally: the angular coupling structure is fixed analytically by the irrep mapping, while the neural network learns the reduced coefficients associated with each orbital pair and chemical environment. This is precisely why the matrix-block prediction problem can be posed naturally in an irrep basis rather than directly in Cartesian matrix entries.

2.4. From equivariant local features to sparse operator prediction

`Mandala` represents each operator $X \in \{H, S, D\}$ as a sparse family of atom-pair blocks $X_{ij}^{\boldsymbol{L}}$, where i and j index atoms and $\boldsymbol{L}$ labels periodic images. The block representation is converted into an irrep representation by a fixed change of basis,

$$\boldsymbol{x}_{ij}^{\boldsymbol{L}} = Q_{a_i a_j} \operatorname{vec}\big(X_{ij}^{\boldsymbol{L}}\big) , \tag{29}$$

where a_i and a_j denote the atomic species of atoms i and j, and $Q_{a_i a_j}$ is the species-pair-specific block-to-irrep transform. The inverse map reconstructs matrix blocks from predicted irrep vectors,

$$\operatorname{vec}\big(X_{ij}^{\boldsymbol{L}}\big) = Q_{a_i a_j}^{-1} \boldsymbol{x}_{ij}^{\boldsymbol{L}}, \tag{30}$$

which, in the orthonormal convention used internally, is implemented as a transpose. In the code these maps are materialized by the `BlockIrrepMapper`. The same component is reused by the data pipeline, the model heads, and the evaluation routines.

Once the network predicts a consistent set of sparse operators, selected quantities are obtained directly from those operators rather than through a separate surrogate model. For sparse block matrices, the trace of a product is evaluated as

$$\operatorname{Tr}(AB) = \sum_{(\boldsymbol{L}, i, j)} \operatorname{tr}\big(A_{ij}^{\boldsymbol{L}} B_{ji}^{-\boldsymbol{L}}\big) , \tag{31}$$

which is exactly the sparse contraction implemented by `Mandala`. This gives direct access to electron counts and band-energy observables from the same predicted operator set. The resulting workflow differs from direct scalar

regression: internal consistency between matrices and observables is built into the model formulation rather than imposed only at the level of post hoc analysis.

3. Numerical Implementation

3.1. Software design goals, sparse data model, and periodic graph construction

`Mandala` is organized as a layered scientific software stack whose core workflow consists of parsing electronic-structure outputs, harmonizing basis conventions, building sparse operators and graphs, predicting irrep vectors, reconstructing operators, and evaluating observables. Figure 1 summarizes the data-loading and sample-construction path. The central data abstractions are `BlockMatrix`, `IrrepsBlockData`, and `Snapshot`. A `Snapshot` contains all matrices and metadata associated with one geometry, including positions and the simulation cell. This keeps Hamiltonian, overlap, and density predictions tied to a single physically consistent basis and sparse indexing scheme.

The sparse operator representation stores blocks by ordered species pair and by explicit edge metadata $(\boldsymbol{L}, i, j)$, where $\boldsymbol{L} \in \mathbb{Z}^3$ is the lattice-image shift. Basis-conversion modules map backend-specific orbital orderings and sign conventions into the common internal representation expected by the equivariant stack. In the current framework, OpenMX, FHI-aims, and PySCF parsers and convention mappers are available [22–25].

Graph construction mirrors the sparse operator indexing. Nodes correspond to atoms, while edges correspond to self-interactions and neighbor interactions inside a cutoff radius. For periodic systems, each edge carries an integer shift vector $\boldsymbol{L}$ and a relative displacement

$$\boldsymbol{r}_{ij}^{\boldsymbol{L}} = \boldsymbol{R}_j - \boldsymbol{R}_i + \sum_{\alpha=1}^{3} L_\alpha \boldsymbol{a}_\alpha, \tag{32}$$

where $\{\boldsymbol{a}_\alpha\}$ are the cell vectors. These displacements are used to construct radial embeddings and spherical-harmonic features. The directed periodic edge set is therefore

$$\mathcal{E} = \left\{ (i, j, \boldsymbol{L}) \;\middle|\; \left\| \boldsymbol{r}_{ij}^{\boldsymbol{L}} \right\|_2 \le r_{\mathrm{c}} \right\}, \tag{33}$$

with a further partition into diagonal, shifted-self, and off-diagonal edges,

$$\mathcal{E}_{\text{diag}} = \{(i, j, \boldsymbol{L}) \in \mathcal{E} \mid i = j,\ \boldsymbol{L} = \boldsymbol{0}\}\,, \tag{34}$$

$$\mathcal{E}_{\text{shifted\_self}} = \{(i, j, \boldsymbol{L}) \in \mathcal{E} \mid i = j,\ \boldsymbol{L} \neq \boldsymbol{0}\}\,, \tag{35}$$

$$\mathcal{E}_{\text{offdiag}} = \{(i, j, \boldsymbol{L}) \in \mathcal{E} \mid i \neq j\}\,. \tag{36}$$

To preserve exact agreement between graph edges and sparse matrix blocks, `Mandala` imposes a canonical ordering of edges by distance and lattice shift, with reverse-edge lookup tables used for sparse traces and symmetrization. This explicit reverse alignment is essential for both physical correctness and reproducibility under periodic boundary conditions.

The same snapshot-level representation preserves access to operator-derived postprocessing. In particular, the predicted Hamiltonian may be used for band-structure analysis within the same sparse-operator workflow through `Snapshot.get_band_structure()`.

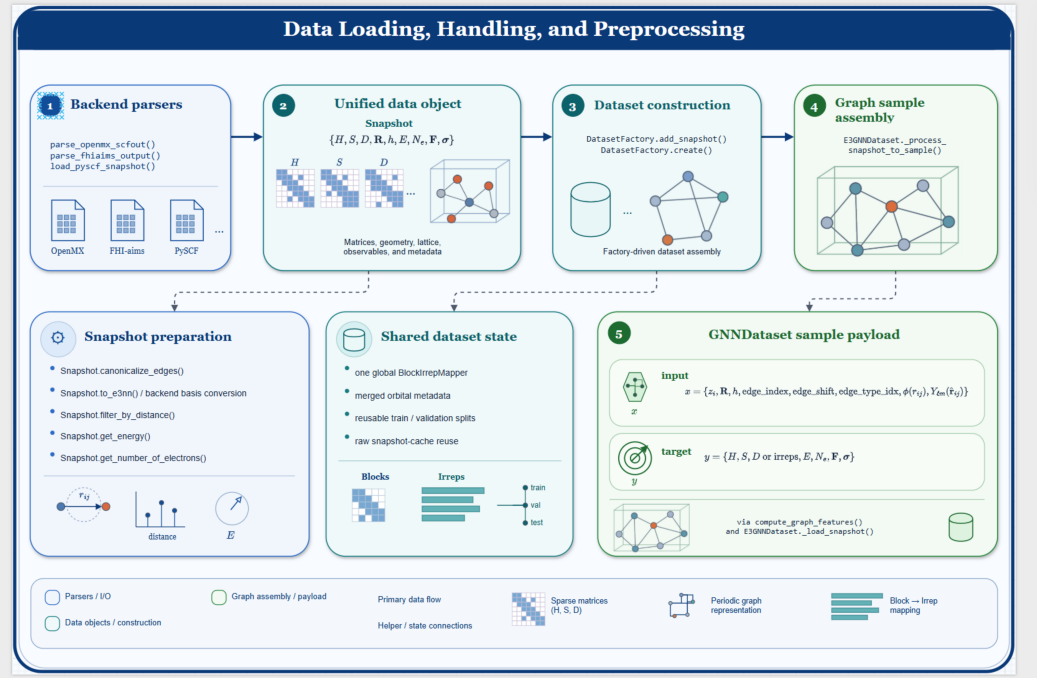

Figure 1: Data loading, harmonization, and graph-sample construction in `Mandala`. Electronic-structure outputs from OpenMX, FHI-aims, and PySCF are converted by backend-specific parsers into a unified `Snapshot` representation containing geometry, periodic-cell information, sparse atom-pair blocks of the Hamiltonian H, overlap S, and density matrix D, and available observable metadata. Snapshot preparation establishes canonical edge ordering, harmonizes backend basis and spherical-harmonic conventions, applies optional distance filtering, and evaluates energy and electron-count targets. Dataset construction is coordinated by `DatasetFactory`; shared state provides a global `BlockIrrepMapper`, merged orbital metadata, reusable data splits, and snapshot cache for performance. `E3GNNDataset` finally assembles graph-aligned samples with atomic species and positions, periodic edge indices and shifts, edge classes, radial embeddings $\phi(r_{ij})$, and spherical harmonics $Y_{\ell m}(\hat{\mathbf{r}}_{ij})$. Targets contain sparse matrix blocks or irrep vectors and, when available, band energy and electron count.

3.2. E(3)-equivariant message passing, readout heads, and architecture variants

The network is composed of node encoders, edge encoders, a stack of equivariant message-passing blocks, and matrix-specific readout heads. Figure 2 summarizes the computational structure and configuration points of the model. We first introduce the base architecture and then describe the additional variants that extend it. If z_i denotes the atomic-species index of

atom i, the initial node representation is

$$\boldsymbol{h}_i^{(0)} = \mathrm{Emb}_{\mathrm{atom}}(z_i) \in d_0 \times 0e, \tag{37}$$

so node encoders initialize scalar chemical features. One available edge-encoder variant first forms an edge-type embedding $\boldsymbol{u}_{ij} = \mathrm{Emb}_{\mathrm{edge}}(t_{ij})$ and a geometric feature $\boldsymbol{g}_{ij} = \varphi(r_{ij}) \oplus Y(\hat{\boldsymbol{r}}_{ij})$, and then applies

$$\boldsymbol{e}_{ij}^{(0)} = \mathrm{TP}_{\mathrm{enc}}(\boldsymbol{u}_{ij}, \boldsymbol{g}_{ij})\,. \tag{38}$$

If $\boldsymbol{h}_i^{(t)}$ denotes node features and $\boldsymbol{e}_{ij}^{(t)}$ denotes edge features at layer t, a generic `Mandala` message-passing step may be written as

$$\boldsymbol{m}_{ij}^{(t)} = \Phi_{\mathrm{edge}}^{(t)}\left(\boldsymbol{h}_i^{(t)}, \boldsymbol{h}_j^{(t)}, \boldsymbol{e}_{ij}^{(t)}, Y(\hat{\boldsymbol{r}}_{ij}), \varphi(r_{ij})\right), \tag{39}$$

$$\boldsymbol{h}_i^{(t+1)} = \Phi_{\mathrm{node}}^{(t)}\left(\boldsymbol{h}_i^{(t)}, \mathrm{Agg}_{j\in\mathcal{N}(i)}\, \boldsymbol{m}_{ij}^{(t)}\right), \tag{40}$$

Here, $\boldsymbol{h}_i^{(t)}$ denotes the irrep-valued latent feature carried by node i at message-passing layer t, $\boldsymbol{e}_{ij}^{(t)}$ denotes the corresponding edge feature for the directed edge $i \to j$, $\boldsymbol{m}_{ij}^{(t)}$ is the edge message constructed at that layer, $\hat{\boldsymbol{r}}_{ij}$ is the unit vector along the relative displacement between atoms i and j, $Y(\hat{\boldsymbol{r}}_{ij})$ denotes the spherical-harmonic embedding of that direction, and $\varphi(r_{ij})$ denotes a radial distance embedding. The neighbor set $\mathcal{N}(i)$ contains the atoms connected to node i within the graph cutoff. In this base form, $\Phi_{\mathrm{edge}}^{(t)}$ and $\Phi_{\mathrm{node}}^{(t)}$ are equivariant maps assembled from tensor products, equivariant linear layers, radial MLPs, parity-aware nonlinearities, and optional residual connections, while Agg denotes a configurable aggregation rule such as summation or attention.

In the concrete implementation, these updates are built from equivariant convolutions with radial weighting. Writing the concatenated local feature as

$$\boldsymbol{f}_{ij}^{(t)} = \boldsymbol{h}_i^{(t)} \oplus \boldsymbol{h}_j^{(t)} \oplus \boldsymbol{e}_{ij}^{(t)}, \tag{41}$$

the tensor-product stage produces

$$\boldsymbol{z}_{ij}^{(t)} = \mathrm{TP}\left(\boldsymbol{f}_{ij}^{(t)}, Y(\hat{\boldsymbol{r}}_{ij})\right), \tag{42}$$

followed by an optional equivariant nonlinearity

$$\widetilde{\boldsymbol{z}}_{ij}^{(t)} = \mathrm{Act}_{\mathrm{eq}}\left(\boldsymbol{z}_{ij}^{(t)}\right), \tag{43}$$

and a radial MLP that returns one scalar coefficient per output irrep block,

$$\boldsymbol{w}_{ij}^{(t)} = \mathrm{MLP}_{\mathrm{rad}}(\varphi(r_{ij}))\,. \tag{44}$$

If $\widetilde{\boldsymbol{z}}_{ij}^{(t,a)}$ denotes the ath output irrep slice, the weighted equivariant convolution is

$$\boldsymbol{q}_{ij}^{(t,a)} = w_{ij}^{(t,a)}\, \widetilde{\boldsymbol{z}}_{ij}^{(t,a)}. \tag{45}$$

The node update then aggregates edge messages as

$$\overline{\boldsymbol{m}}_i^{(t)} = \sum_{j \in \mathcal{N}(i)} \boldsymbol{q}_{ji}^{(t)}, \tag{46}$$

while edge and node updates both admit optional self-connections and residual paths. In the sequential message block used by the code, node features are first updated from the current edges and only then are edge features updated using the new node state.

The readout stage maps final latent features to the irrep content required by each matrix target. For a target operator X, the head predicts irrep vectors

$$\widehat{\boldsymbol{x}}_{ij}^{\boldsymbol{L},X} = \Psi_X\Big(\boldsymbol{e}_{ij}^{(L)}, \boldsymbol{h}_i^{(L)}, \boldsymbol{h}_j^{(L)}, \varphi(r_{ij})\Big)\,, \tag{47}$$

followed by inverse irrep-to-block conversion

$$\widehat{X}_{ij}^{\boldsymbol{L}} = Q_{a_i a_j}^{\top} \widehat{\boldsymbol{x}}_{ij}^{\boldsymbol{L},X}. \tag{48}$$

For richer readout variants, the edge used by the head may first be transformed through a tensor-square map,

$$\boldsymbol{t}_{ij}^{(L)} = \mathrm{TS}\Big(\boldsymbol{e}_{ij}^{(L)}\Big)\,, \tag{49}$$

with the actual head input defined by

$$\boldsymbol{u}_{ij}^{(L)} = \begin{cases} \boldsymbol{t}_{ij}^{(L)}, & \text{tensor-square head enabled,} \\ \boldsymbol{e}_{ij}^{(L)}, & \text{otherwise.} \end{cases} \tag{50}$$

The head can then branch by edge class,

$$\widehat{\boldsymbol{x}}_{ij}^{\boldsymbol{L},X} = \begin{cases} \Psi_{\mathrm{diag}}^{X}(\boldsymbol{u}_{ij}^{(L)}), & (i,j,\boldsymbol{L}) \in \mathcal{E}_{\mathrm{diag}}, \\ \Psi_{\mathrm{shifted\_self}}^{X}(\boldsymbol{u}_{ij}^{(L)}), & (i,j,\boldsymbol{L}) \in \mathcal{E}_{\mathrm{shifted\_self}}, \\ \Psi_{\mathrm{offdiag}}^{X}(\boldsymbol{u}_{ij}^{(L)}), & (i,j,\boldsymbol{L}) \in \mathcal{E}_{\mathrm{offdiag}}. \end{cases} \tag{51}$$

When the onsite node-embedding override is enabled, zero-shift self-edges use

$$\boldsymbol{u}_{ii}^{\mathbf{0}} \leftarrow \boldsymbol{h}_i^{(L)} \tag{52}$$

instead of the corresponding edge embedding. The most elaborate readout variants further split scalar and non-scalar output channels or factorize each predicted block into a normalized equivariant direction and a positive scalar magnitude. In the latter case one predicts

$$\widehat{\boldsymbol{x}}_{e,\mathrm{norm}}^{X} = \Psi_{\mathrm{dir}}^{X}(\boldsymbol{u}_e), \qquad \widehat{m}_e^{X} = \exp\!\big(\mathrm{MLP}_{\mathrm{mag}}^{X}\big(L_{\mathrm{mag}}^{X}(\boldsymbol{t}_e) \oplus \varphi(r_e)\big)\big)\,, \tag{53}$$

and reconstructs the actual block prediction as

$$\widehat{X}_e^{\mathrm{actual}} = \widehat{m}_e^{X}\, \widehat{X}_e^{\mathrm{norm}}. \tag{54}$$

Because heads share the same equivariant latent representation, `Mandala` can predict Hamiltonian, overlap, and density matrices jointly with only modest additional readout cost.

Starting from this base architecture, `Mandala` exposes a broad family of variants within the same software interface. Search dimensions include the hidden irrep content, the angular cutoff $\ell_{\max}$, the number of message-passing layers, the radial basis size, the depth of the neck and head MLPs, and the choice of tensor product implementation. The message-passing blocks support alternative edge updates (tensor-product, concatenation-based, and replacement-style variants), alternative node updates (sum, concatenation, and replacement variants), alternative aggregation mechanisms (including attention-style aggregation), pre- and post-linear transformations, optional self-connections, and optional node and edge residual paths.

The encoder and head stages are likewise configurable. Edge encoders support various variants, together with optional tensor-square feature expansion of spherical-harmonics. Readout heads support direct equivariant projections, deeper equivariant MLP heads, optional tensor-square preconditioning of hidden features, scalar-specific auxiliary branches, optional log-scale or magnitude factorization for output amplitudes, optional use of node embeddings for self-edges, and separate treatment of diagonal, shifted-self, and off-diagonal blocks. Because all of these variants share the same block-sparse data model and irrep mapping layer, architecture search can be performed without changing the surrounding training and evaluation workflow.

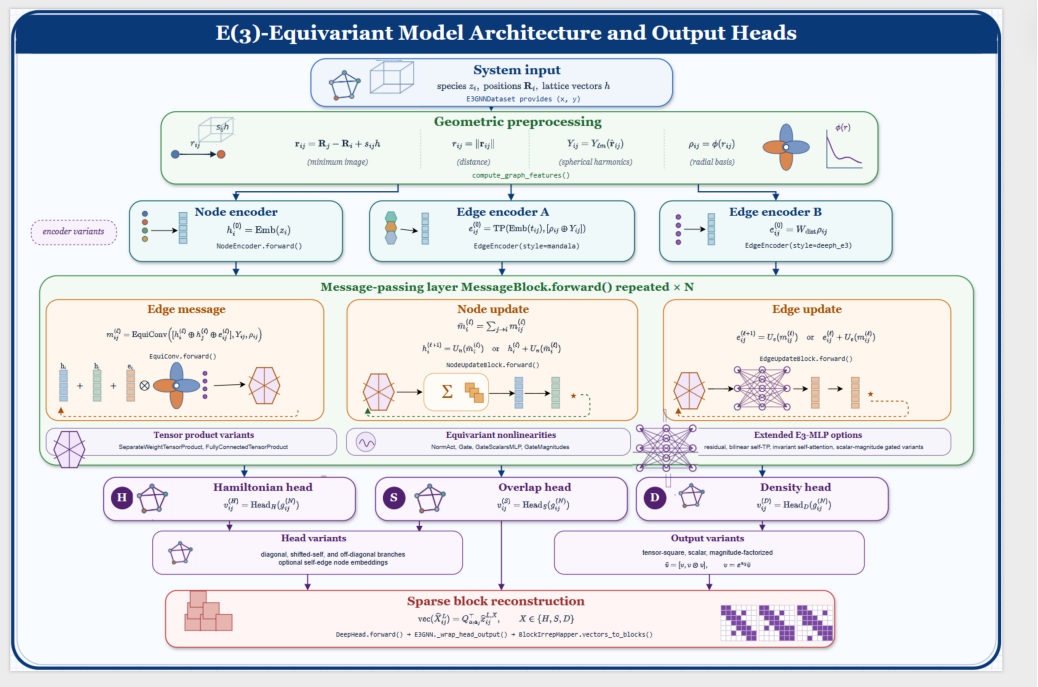

Figure 2: E(3)-equivariant message-passing architecture and sparse-operator readout. Atomic species, positions, and lattice vectors are transformed into periodic relative displacements, distances, radial features $\phi(r_{ij})$, and real spherical harmonics $Y_{\ell m}(\widehat{\mathbf{r}}_{ij})$. Node and alternative edge encoders initialize irrep-valued latent features. Each repeated `MessageBlock` constructs equivariant edge messages from source-node, target-node, edge, radial, and angular features; aggregates the messages to update nodes; and updates edge features, with optional residual and self-connection paths. The implementation supports separate-weight or fully connected tensor products, several equivariant nonlinearities, and extended E(3)-MLP variants. Operator-specific Hamiltonian, overlap, and density heads map the final latent representation to output irreps, with optional tensor-square, scalar, magnitude-factorized, and edge-class-specific branches. The `BlockIrrepMapper` then converts the predicted irrep vectors into aligned sparse blocks $\widehat{H}$, $\widehat{S}$, and $\widehat{D}$.

3.3. Sparse operator algebra, symmetrization, gauge correction, and density normalization

Sparse operator manipulations are implemented directly on aligned block data. Given two sparse operators A and B with reverse-edge alignment already established, `Mandala` evaluates their trace contraction as

$$\mathrm{Tr}(AB) = \sum_{(\boldsymbol{L},i,j)} \mathrm{tr}\left(A_{ij}^{\boldsymbol{L}} B_{ji}^{-\boldsymbol{L}}\right), \tag{55}$$

without assembling dense global matrices. This operator algebra is reused for energy and electron-count evaluation, loss construction, and diagnostic analysis. For efficient vectorized evaluation, the implementation precomputes a reverse-edge alignment permutation. For each pair key k with reverse key k^{rev}, the permutation P_k is defined by

$$P_k(n) = m \quad \Longleftrightarrow \quad \big(\boldsymbol{L}_{k,n}, i_{k,n}, j_{k,n}\big) = \big(-\boldsymbol{L}_{k^{\mathrm{rev}},m}, j_{k^{\mathrm{rev}},m}, i_{k^{\mathrm{rev}},m}\big), \tag{56}$$

so that the aligned trace may be written as

$$\mathrm{Tr}(AB) = \sum_k \sum_n \mathrm{tr}\big(A_{k,n}\, B_{k^{\mathrm{rev}},P_k(n)}\big)\,. \tag{57}$$

For matrix targets expected to be symmetric or Hermitian in the chosen basis, `Mandala` supports explicit sparse symmetrization after block reconstruction. In the real-valued convention used by the implementation, this takes the form

$$X_{ij}^{\boldsymbol{L}} \leftarrow \frac{1}{2}\left(X_{ij}^{\boldsymbol{L}} + \big(X_{ji}^{-\boldsymbol{L}}\big)^{\top}\right). \tag{58}$$

This operation is carried out using precomputed reverse-edge permutations, so that symmetry is enforced consistently across periodic images and self-edges.

Hamiltonian gauge freedom is handled operationally through the overlap-shift correction described in Section 2.1.1. In practical evaluation and analysis workflows, `Mandala` computes

$$\widetilde{H}^{\mathrm{pred}} = H^{\mathrm{pred}} - \mu_H S, \qquad \mu_H = \frac{\big\langle H^{\mathrm{pred}} - H^{\mathrm{ref}}, S\big\rangle_F}{\langle S, S\rangle_F}, \tag{59}$$

so that purely gauge-like Hamiltonian offsets are removed before reporting gauge-corrected operator errors.

The framework also supports density normalization to the correct number of electrons. If a predicted density $\widehat{D}$ and overlap $\widehat{S}$ produce $\widehat{N}_e = \mathrm{Tr}(\widehat{D}\widehat{S})$, then a normalized density is obtained by the scalar rescaling

$$\widehat{D}_{\mathrm{norm}} = \beta \widehat{D}, \qquad \beta = \frac{N_e^{\mathrm{target}}}{\mathrm{Tr}(\widehat{D}\widehat{S})}. \tag{60}$$

This provides a simple and effective way to enforce electron-count consistency.

Together, sparse trace evaluation, symmetry restoration, gauge correction, and density normalization define a compact postprocessing layer. It turns raw network outputs into physically interpretable operators ready for observable evaluation and Hamiltonian-based analyses such as band-structure calculation.

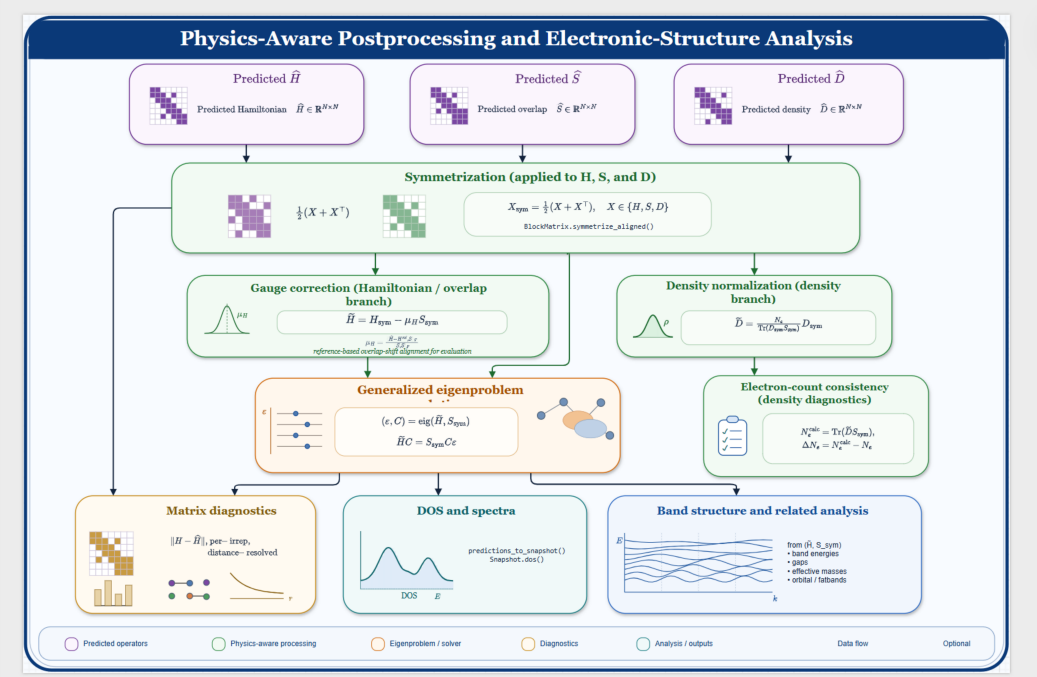

Figure 3: Physics-aware postprocessing and electronic-structure analysis of predicted sparse operators. The predicted Hamiltonian $\widehat{H}$, overlap $\widehat{S}$, and density matrix $\widehat{D}$ are first symmetrized using reverse-edge-aligned sparse blocks. The Hamiltonian branch removes an arbitrary overlap-proportional energy offset through the gauge correction $\widetilde{H} = H_{\mathrm{sym}} - \mu_H S_{\mathrm{sym}}$. The density branch rescales D_{sym} to enforce the target electron count through $\widetilde{D} = N_e D_{\mathrm{sym}} / \mathrm{Tr}(D_{\mathrm{sym}} S_{\mathrm{sym}})$ and reports the remaining electron-count discrepancy. The corrected Hamiltonian and overlap define the generalized eigenproblem $\widetilde{H}C = S_{\mathrm{sym}} C\varepsilon$. The resulting physically processed operators support block-, irrep-, and distance-resolved matrix diagnostics, density-of-states and spectral calculations, and band-structure and related analyses while retaining the sparse representation.

3.4. Observable guidance with band energies and electron counts

`Mandala` supports pure matrix learning, mixed matrix-and-observable learning, and fully observable-guided training from one common operator-prediction pipeline. Figure 4 summarizes the corresponding training flow.

For a chosen set of matrix targets $\mathcal{M} \subseteq \{H, S, D\}$, the general training objective is

$$\mathcal{L} = \sum_{X \in \mathcal{M}} \lambda_X \mathcal{L}_X^{\text{matrix}} + \lambda_E \mathcal{L}^E + \lambda_N \mathcal{L}^N. \tag{61}$$

The matrix terms compare predicted and reference sparse operators, typically by a weighted combination of ℓ_1 and ℓ_2 errors,

$$\mathcal{L}_X^{\text{matrix}} = \eta \, \text{MAE}(\widehat{X}, X) + (1 - \eta) \, \text{MSE}(\widehat{X}, X), \tag{62}$$

with η controlled by configuration.

Observable guidance is defined on quantities derived from the predicted operators. In the operator-centered workflow used throughout `Mandala`, the band energy and the number of electrons are

$$\widehat{E_b} = \text{Tr}(\widehat{D}\widehat{H}), \qquad \widehat{N}_e = \text{Tr}(\widehat{D}\widehat{S}), \tag{63}$$

which gives

$$\mathcal{L}^E = \text{MSE}(\widehat{E_b}, E^{\text{ref}}), \qquad \mathcal{L}^N = \text{MSE}(\widehat{N}_e, N_e^{\text{ref}}). \tag{64}$$

In the sparse aligned implementation, these quantities are evaluated as

$$\widehat{E_b} = \sum_k \sum_n \text{tr}\Big(\widehat{H}_{k,n} \, \widehat{D}_{k^{\text{rev}}, P_k(n)}\Big), \tag{65}$$

$$\widehat{N}_e = \sum_k \sum_n \text{tr}\Big(\widehat{D}_{k,n} \, \widehat{S}_{k^{\text{rev}}, P_k(n)}\Big). \tag{66}$$

The framework also supports mixed observable guidance paths in which one of the operators entering the trace is replaced by the reference operator, for example the diagnostic contractions

$$\widehat{E}_{H|D^{\text{ref}}} = \text{Tr}\Big(\widehat{H} D^{\text{ref}}\Big), \qquad \widehat{E}_{D|H^{\text{ref}}} = \text{Tr}\Big(H^{\text{ref}} \widehat{D}\Big), \tag{67}$$

and

$$\widehat{N}_{D|S^{\text{ref}}} = \text{Tr}\Big(\widehat{D} S^{\text{ref}}\Big), \qquad \widehat{N}_{S|D^{\text{ref}}} = \text{Tr}\Big(D^{\text{ref}} \widehat{S}\Big), \tag{68}$$

enabling diagnostic and training modes that separate errors due to different predicted operators.

The current implementation also exposes experimental derivatives of the operator-derived band energy with respect to atomic coordinates and the cell,

$$\boldsymbol{F}_I^{\mathrm{band}} = -\frac{\partial \widehat{E_b}}{\partial \boldsymbol{R}_I}, \qquad \sigma^{\mathrm{band}} = \frac{1}{\Omega} h^\top \frac{\partial \widehat{E_b}}{\partial h}, \qquad \Omega = \det(h). \tag{69}$$

These derivatives are not total-energy forces or stresses because $\widehat{E}_b = \mathrm{Tr}(\widehat{D}\widehat{H})$ omits the remaining contributions to the Kohn–Sham total energy. They are therefore not presented here as validated supervision targets.

The supported observable-guidance objective in this work is thus restricted to matrix entries, the band-energy observable, and the electron count.

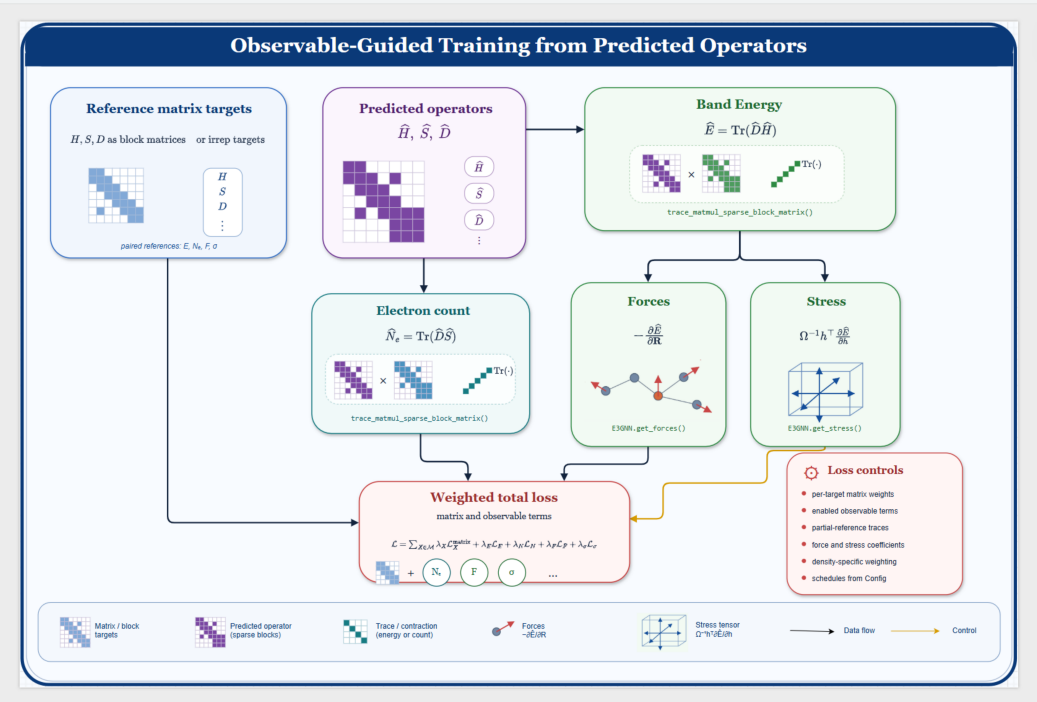

Figure 4: Observable-guided training from predicted sparse operators. Reference Hamiltonian, overlap, and density targets, represented as aligned sparse blocks or irrep coefficients, directly supervise the predicted operators $\widehat{H}$, $\widehat{S}$, and $\widehat{D}$. The same operator set yields the band-energy observable $\widehat{E}_b = \mathrm{Tr}(\widehat{D}\widehat{H})$ and electron count $\widehat{N}_e = \mathrm{Tr}(\widehat{D}\widehat{S})$ through differentiable sparse trace contractions. Experimental force and stress outputs are derivatives of the band energy only and are not total-energy derivatives. All supported matrix and observable terms are combined in a weighted objective. Configuration controls select per-target matrix weights, observable terms, partial-reference traces, density-specific weighting, and training schedules, allowing pure matrix supervision or joint matrix-, band-energy-, and electron-count-guided training without separate observable heads.

3.5. Configuration-driven architecture search

Because the same sparse operator workflow supports many architectural choices, `Mandala` treats model search as a primary scientific use case rather than as an auxiliary engineering convenience. All major representation, encoder, message-passing, readout, and loss-weighting choices are exposed as structured configuration parameters and can therefore be swept systematically.

Search dimensions include hidden irreps, angular cutoff, radial basis size, message-passing depth, encoder style, tensor-product type, normalization

and nonlinearity families, node and edge update variants, residual pathways, neck and head depth, separate treatment of shifted-self blocks, tensor-square readout options, scalar-specific branches, observable-loss coefficients, and the subset of matrix targets predicted jointly. This wide configuration surface is one of the main scientific strengths of the framework: it enables controlled comparisons between architecture families without forcing changes to the data model, parser layer, or observable-evaluation stack. The importance of systematic hyperparameter exploration in machine-learned electronic-structure models has also been emphasized in related work on training-free optimization strategies [26].

3.6. Current limitations and development roadmap

The present implementation is designed for localized-orbital electronic-structure data that can be represented by real-valued, atom-pair-resolved sparse blocks. Orbital definitions must be known for every chemical species. Locality is controlled through a finite cutoff radius; consequently, accuracy and computational cost depend on whether the selected cutoff captures the relevant operator range. Periodic spectral analysis further requires consistent lattice-shift support and a numerically well-conditioned overlap matrix.

The current training and logging workflows are primarily optimized for one structure per batch which is typical for these kinds of networks, and larger-batch and distributed-data behavior requires additional validation. The configuration surface is intentionally broad, but not every combination is equally mature; the controlled ablations reported in this work are therefore important for identifying supported reference settings. Most importantly, the scalar $\mathrm{Tr}(DH)$ is a band-energy observable rather than the complete Kohn–Sham total energy. Its coordinate and cell derivatives cannot be interpreted as total-energy forces and stresses. Adding all total-energy contributions and validating their derivatives is a planned extension.

3.7. Verification, reproducibility, and extensibility

`Mandala` includes tests and validation routines covering data parsing, basis harmonization, sparse operator alignment, block-to-irrep mapping, graph construction, equivariant tensor-product paths, observable evaluation, and end-to-end training workflows. These checks are especially important in a code base where subtle mistakes in basis ordering, reverse-edge alignment, or irrep bookkeeping can produce apparently plausible but physically inconsistent results.

Reproducibility follows from configuration-driven runs, explicit snapshot serialization, deterministic sparse edge ordering, and checkpointed training. Extensibility follows from the layered structure described above: new electronic structure backends, new observable definitions, new equivariant blocks, and new training objectives can be incorporated without changing the core operator data model. This design allows `Mandala` to function both as a reusable scientific software package and as a framework for rapid methodological experimentation.

4. Results and ablations

4.1. ZnCuSnSeS envelope-factorization ablation

We performed a controlled ablation using perturbed ZnCuSnSeS structures. The baseline predicts Hamiltonian blocks directly (`hamiltonian_envelope_mode=off`), while the treatment multiplies the predicted blocks by a fitted pair-distance envelope (`multiply_prediction`). Both treatments use seeds 41–45 and the same fixed data split. The ten runs share the split hash recorded with the seed-level source data distributed alongside the paper; the run records are also available from the W&B sweep.

The median validation Hamiltonian MAE decreases from 0.005221 Ha without the envelope to 0.004930 Ha with prediction factorization, a relative reduction of 5.6%. The corresponding means are 0.005167 and 0.004941 Ha, respectively, a 4.4% reduction. The treatment improves four of the five paired seeds. The mean paired difference (treatment minus baseline) is -2.26×10^{-4} Ha, with a 95% t interval from -4.86×10^{-4} to 3.37×10^{-5} Ha. Thus, the completed experiment provides encouraging evidence for the envelope factorization, while the interval and small number of seeds motivate a restrained interpretation rather than a claim of definitive statistical separation.

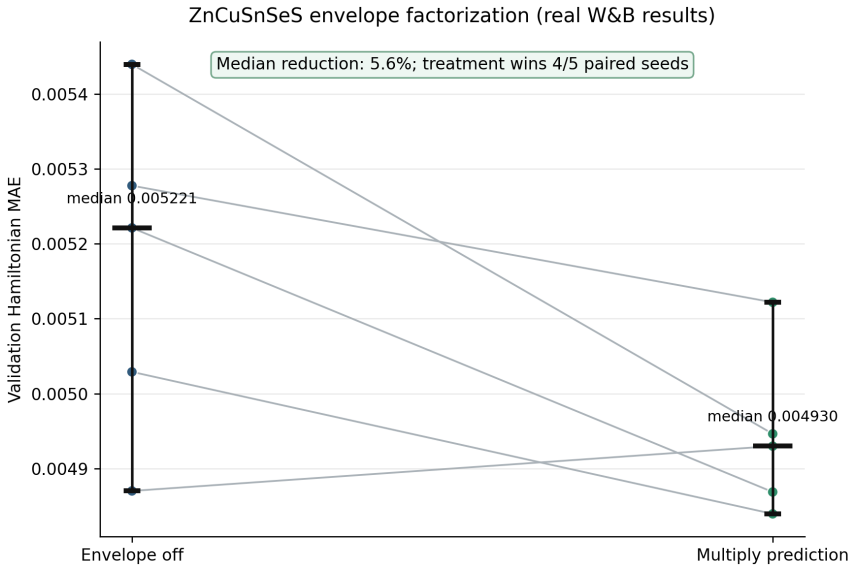

Figure 5: Results for the ZnCuSnSeS envelope-factorization ablation. Colored points show the final W&B validation Hamiltonian MAE for seeds 41–45, and gray lines connect paired seeds. Horizontal black marks denote medians; vertical black intervals are percentile bootstrap intervals for the median. The fitted envelope reduces the median error by 5.6% and wins four of five paired runs.

4.2. Other ablations

PLACEHOLDER — SYNTHETIC DATA — NOT SCIENTIFIC RESULTS

1. SiO_x energy guidance: `loss_coef_observables=0` versus `0.003`, with Hamiltonian and band-energy validation errors.
2. Mature ZnCuSnSeS spectral fine-tuning: `spectral_loss_coef=0` versus `0.001`, with Hamiltonian and spectral validation errors. Duplicate sweep instances will be consolidated before analysis.
3. ZnCuSnSeS node aggregation: `sum` versus `attention`.
4. ZnCuSnSeS shifted-self handling: `separate_shifted_self=false` versus `true`.

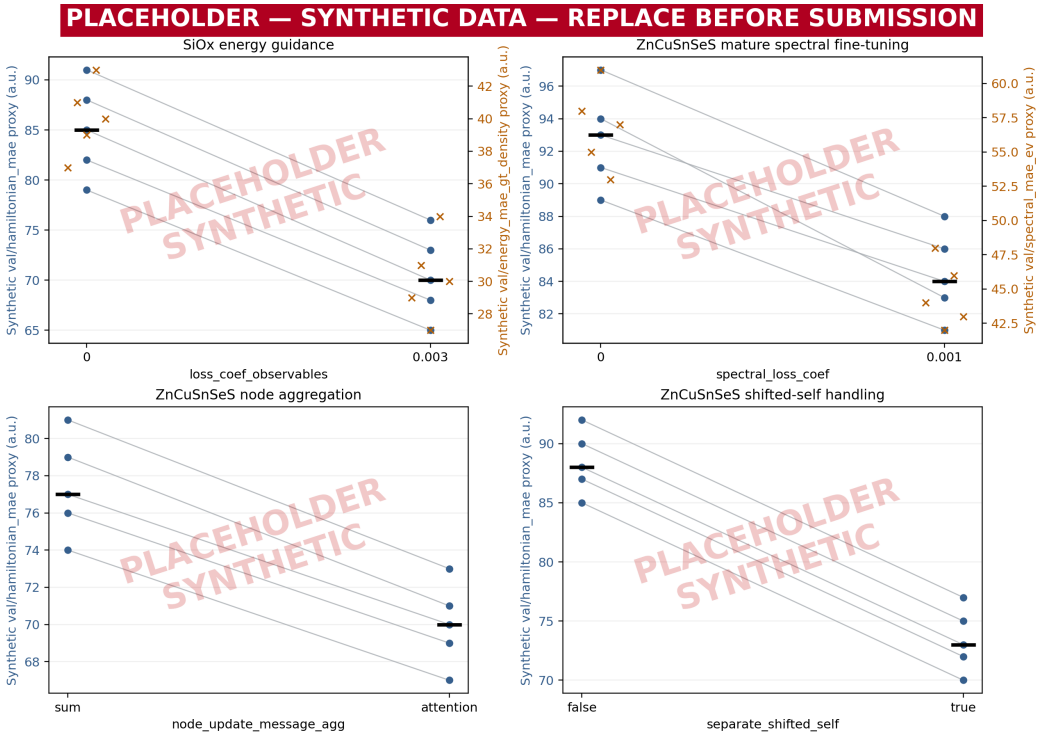

Figure 6: **PLACEHOLDER — SYNTHETIC DATA — NOT RESULTS.** Layout preview for four running ablations. Every displayed value is fabricated in arbitrary units solely to reserve the intended paired-seed presentation. The final plots will use five paired seeds per treatment and the indicated W&B validation metrics.

5. Public Interface and Usage Examples

The public interface keeps user-facing code deliberately lightweight: simple scripts allow users to run extensive analyses. A calculation can be loaded and its sparse operator-derived quantities examined directly:

```
from data.snapshot import Snapshot

snapshot = Snapshot.from_openmx(
    "sample.matrix", "sample.info.out", convention="e3nn"
)
print(snapshot.get_energy())
print(snapshot.get_number_of_electrons())
```

A trained model is restored with its saved configuration and orbital metadata, then applied directly to the same snapshot:

```
import torch
from net.evaluation import predict_snapshot, restore_model_for_snapshot

device = "cuda" if torch.cuda.is_available() else "cpu"
model = restore_model_for_snapshot("best_model.pt", snapshot, device=
    device)
predictions = predict_snapshot(model, snapshot)
predicted_hamiltonian = predictions["hamiltonian"]
```

One can evaluate a Hamiltonian-only model on band energy using the reference density:

```
from core.sparse_math import trace_matmul_sparse_block_matrix

band_energy = trace_matmul_sparse_block_matrix(
    predicted_hamiltonian, snapshot.density
)
```

Likewise, the predicted Hamiltonian can be combined explicitly with a reference overlap matrix for generalized-eigenvalue band-structure analysis:

```
from net.evaluation import predictions_to_snapshot

predicted = predictions_to_snapshot(
    predictions, snapshot, reference_matrices=("overlap", "density")
)
bands = predicted.get_band_structure(
    path="GXWKG",
    npoints=200,
```

```
)
print(bands.eigenvalues.shape)
```

This interface supports a wide range of workflows, with minimal user code required to load data, restore models, and evaluate observables.

6. Conclusion

`Mandala` is a scientific software framework for learning sparse electronic-structure operators with E(3)-equivariant graph neural networks. Its defining features are the explicit sparse block representation of Hamiltonian, overlap, and density matrices; the modular software layers that connect parsing, basis harmonization, irrep mapping, graph construction, model definition, and training; and the ability to supervise not only matrices themselves but also observables derived from them.

The framework is designed to connect two communities that are often separated in practice. From the electronic-structure side, it retains access to operator-level information and basis-aware quantities. From the atomistic machine-learning side, it inherits the flexibility of modern equivariant message passing, multitask learning, and automated hyperparameter search. By combining these perspectives, `Mandala` supports workflows in which one can train on matrix fidelity, band-energy-observable fidelity, and electron-count fidelity within one coherent implementation.

In this sense, `Mandala` is aimed at simulation settings that go beyond conventional machine-learning interatomic potentials by retaining access to the predicted electronic structure, including Hamiltonian-derived quantities such as band structure and related observables. This makes the framework particularly relevant for problems in which explicitly electronic effects, such as charge transfer or electronically driven changes in bonding, are part of the physics that must be modeled rather than treated only indirectly. In its current form, `Mandala` is positioned as a modular platform for operator-level machine learning in electronic-structure simulation and for systematic exploration of equivariant architectures built on that operator-centered representation.

Acknowledgments

This work was partially supported by the Center for Advanced Systems Understanding (CASUS), financed by the German Federal Ministry of Ed-

ucation and Research and by the Saxon state government out of the state budget approved by the Saxon State Parliament, and by the University of Wrocław. Computational resources were provided by the `hemera` and `rosi` clusters of HZDR.

Appendix A. Configuration Inventory

This appendix summarizes the current Mandala configuration surface.

Appendix A.1. Geometry, Basis, and Graph Construction

`cutoff_radius` Default: `11.0`. Global neighbor cutoff used for graph construction and target filtering.

`l_max` Default: `6`. Maximum angular momentum used for spherical harmonics and auto-built hidden irreps.

`hidden_base_dim` Default: `64`. Base multiplicity for the auto-built hidden irreps at `l = 0`.

`hidden_irreps` Default: an explicit representation through $l = 6$: `128x0e+16x0o+8x1e+64x1o+24x2e+8x2o+8x3e+24x3o+16x4e+4x4o+4x5e+12x5o+8x6e`. Setting this to `None` enables automatic construction.

`emb_use_odd_features` Default: `True`. Whether auto-built hidden irreps include odd-parity channels.

`edge_type_emb_dim` Default: `32`. Dimension of the learned scalar embedding for edge types.

`edge_encoder_style` Default: `rich`. Values: `rich` uses edge-type, radial, and spherical-harmonic features, while `distance` uses a distance-only scalar projection.

`edge_encoder_use_sh_tensor_square` Default: `True`. Whether to apply tensor square to spherical-harmonic features before combination.

`e3layernorm` Default: `False`. Enables equivariant layer normalization after encoder and update blocks.

`n_radial` Default: `128`. Number of radial basis channels in `soft_one_hot_linspace(...)`.

`radial_layers` Default: `(128, 128, 128)`. Hidden layer widths of the radial MLP inside `EquiConv`.

`pair_conditioned_radial_mlp` Default: `False`. Enables pair-conditioned radial weights.

`pair_distance_normalization` Default: `off`; `pair_r0` normalizes distances with fitted pair scales.

`radial_embedding_scale` Default: `none`. Optional scale applied to the radial basis embedding.

`apply_cutoff_to_targets` Default: `True`. Whether target sparse matrices are filtered to the same cutoff as the graph inputs.

`require_exact_edge_match` Default: `True`. Whether graph-target reconciliation must be exact.

`precompute_edge_features` Default: `True`. Whether edge radial embeddings and spherical harmonics are cached in the sample payload.

Appendix A.2. Message Passing and Tensor Products

`num_layers_gnn` Default: `2`. Number of message-passing blocks.

`tp_type` Default: `separate_weight`. Values: `separate_weight` uses the internal separate-weight tensor product, and `fully_connected` uses e3nn's implementation.

`use_self_connection` Default: `False`. Enables learned scalar-conditioned skip/self-connection tensor products in node and edge updates.

`edge_update_residual` Default: `False`. Adds residual connection to the edge update output when dimensions match.

`node_update_residual` Default: `False`. Adds residual connection to the node update output when dimensions match.

`edge_update_node_combine` Default: `concat`. Values: `concat` concatenates source and destination node features as a direct sum, `sum` requires matching irreps and adds them, and `tensor_product` learns an equivariant projection from the node pair to the edge-update input space.

`node_update_message_agg` Default: `attention`. Values: `sum` adds incoming messages, `average` divides the sum by the number of incoming edges, and `attention` uses scalar multi-head attention weights with per-head equivariant values.

`node_update_attention_scalar_dim` Default: `64`. Total scalar query/key width used by attention aggregation.

`node_update_attention_heads` Default: `4`. Number of attention heads used when `node_update_message_agg="attention"`.

Appendix A.3. Readout Heads and Output Structure

`neck_depth` Default: `2`. Depth of the shared per-class trunk before matrix-specific projections.

`internal_e3mlp_layers` Default: `1`. Additional equivariant refinement depth inside node and edge update blocks after the main convolution/update.

`head_e3mlp_layers` Default: `1`. Depth of the final matrix-specific equivariant projection MLPs in the heads.

`head_use_node_embeddings_for_self_edges` Default: `True`. For zero-shift diagonal self edges, chooses node features instead of edge features as head input.

`separate_shifted_self` Default: `True`. Splits shifted-self edges into their own readout branch.

`head_use_tensor_square` Default: `False`. Applies `TensorSquare` before branch-specific projection heads.

`head_pair_mode` Default: `split`. Values: `split` and `shared_conditioned`.

`head_diag_output_scale` Default: `1.0`. Post-scale applied to diagonal and shifted-self head outputs.

`head_offdiag_output_scale` Default: `1.0`. Post-scale applied to off-diagonal head outputs.

`head_use_mlp_log_scale` Default: `False`. Enables branch-specific learned log-amplitude scaling MLPs.

`head_log_scale_mlp_n_layers` Default: `1`. Depth of the scalar log-scale MLPs used when `head_use_mlp_log_scale=True`.

Appendix A.4. E(3)-MLP Variants, Activations, and Normalization

Main variant selectors.

`e3mlp_variant` Default: `film`. Values: `basic` is the plain linear-plus-nonlinearity path; `normact` uses normalization before activation; `gate` uses gated equivariant features; `gatemagnitudes` gates by vector magnitudes; `film` conditions blocks with invariant FiLM parameters; `resnormact` and `resgatemagnitudes` add residual-style behavior; `bilinear` uses bilinear self tensor products.

`internal_e3mlp_variant` Default: `resnormact`. Values mirror `e3mlp_variant`; this overrides the refinement MLPs inside message passing.

`head_e3mlp_variant` Default: `normact`. Values mirror `e3mlp_variant`; this overrides the projection MLPs inside the readout heads.

Variant block parameters.

`e3mlp_output_scale` Default: `1.0`. Output scale passed into scaled linear layers used by non-basic E3MLP variants.

`e3mlp_weight_init_scale` Default: `1.0`. Multiplicative scale on initial weights for variant blocks.

`e3mlp_residual_scale` Default: `0.25`. Initial layer-scale or residual strength for residual-style variant blocks.

`e3mlp_pre_norm` Default: `False`. Whether variant blocks use pre-normalization by `EquivariantRMSNorm`.

`e3mlp_norm_eps` Default: `1e-8`. Numerical epsilon for `EquivariantRMSNorm`.

`e3mlp_film_hidden_dim` Default: `64`. Hidden width of the invariant FiLM MLP.

Activation and equivariant nonlinearity family.

`nonlin_kind` Default: `normact`. Values: `normact` applies normalized scalar gating, `s2act` uses e3nn's spherical activation, `gate_scalars_mlp` gates scalars with a learned invariant MLP, and `gate_magnitudes` gates by vector magnitudes.

`activation_scalar` Default: `leakyrelu`. Values: standard scalar activations such as `silu`, `relu`, `gelu`, `tanh`, `sigmoid`, `leakyrelu`, `softplus`, and `softsign`.

`activation_gate` Default: `softplus`. Values: the same scalar activation family is allowed, but this choice is used for gate channels.

`activation_odd_scalar` Default: `tanh`. Values: `tanh` and `identity` are the intended stable choices for odd-parity scalar channels.

`activation_odd_gate` Default: `tanh`. Values: `tanh` and `identity` are the intended stable choices for odd-parity gate scalars.

`s2act_res` Default: `128`. Positive integer controlling the angular resolution used by `S2Activation`.

`norm_kind` Default: `component`. Values: `component` normalizes by component variance, while `norm` normalizes by vector norm.

Appendix A.5. Prediction Targets, Observable Guidance, and Physics-Aware Options

`matrix_targets` Default: `[hamiltonian]`. Values: `hamiltonian`, `overlap`, and `density`. These choose which sparse operators the model predicts.

`train_target` Default: `matrix`. Values: `matrix` applies supervision in block-matrix space, while `irreps` applies it in irrep-vector space.

`partial_train` Default: `None`. Values: `diag` restricts loss to diagonal/self edges, `shifted_self` keeps shifted self-edges, and `offdiag` restricts training to off-diagonal edges.

`train_on_irrep_parts` Default: `False`. When enabled, matrix-space loss is computed after splitting predictions and targets into irrep components.

`enable_energy` Default: `True`. Boolean switch for band-energy observable evaluation and logging.

`enable_num_electrons` Default: `True`. Boolean switch for electron-count observable evaluation and logging.

`enable_forces` Default: `False`. Experimental switch for derivatives of the band energy; these outputs are not total-energy forces.

`enable_stress` Default: `False`. Experimental switch for cell derivatives of the band energy; these outputs are not total-energy stresses.

`train_on_energy` Default: `False`. Boolean switch for adding a band-energy loss term.

`train_on_num_electrons` Default: `False`. Boolean switch for adding an electron-count loss term.

`train_on_forces` Default: `False`. Internal experimental control; it is not part of the validated paper-facing objective.

`train_on_stress` Default: `False`. Internal experimental control; it is not part of the validated paper-facing objective.

`train_observables_on_gt` Default: `True`. Boolean switch. When enabled, observable losses may mix predicted and ground-truth operators.

`log_partial_gt_observables` Default: `True`. Boolean switch for logging hybrid observable diagnostics even when not training on them.

`symmetrize_output` Default: `False`. Boolean switch for symmetrizing predicted block matrices before matrix-space loss computation.

`symmetrize_hamiltonian_targets` Default: `True`. Boolean switch for symmetrizing Hamiltonian targets during dataset preprocessing.

`rescale_density_to_num_electrons` Default: `False`. Boolean switch for rescaling predicted density matrices before artifact evaluation.

`observable_loss_kind` Default: `mae`. Values: `mae` and `mse`.

`hamiltonian_envelope_mode` Default: `off`. Values: `off`, `normalize_target`, and `multiply_prediction`.

`loss_weighting_mode` Default: `off`. Selects optional block-loss weighting, including envelope-based modes.

`spectral_loss_enabled` Default: `False`. Enables differentiable eigenvalue guidance from a configured k-point mesh.

`spectral_loss_coef` Default: `0.0`. Weight of the spectral objective.

`spectral_loss_kmesh` Default: `1x1x1`. k-point mesh used by spectral guidance.

Appendix A.6. Optimization, Regularization, and Training Stability

`lr` Default: `0.01`. Learning rate for `AdamW`.

`max_epochs` Default: `1000`. Intended training horizon.

`max_wall_clock_seconds` Default: `43200` (12 hours). Wall-clock training budget; `None` disables it.

`batch_size` Default: `1`. Batch size for dataset loaders.

`accumulate_grad_batches` Default: `1`. Gradient accumulation factor.

`dropout` Default: `0.0`. Equivariant dropout on all irrep coefficients.

`l1_reg_coef` Default: `0.0`. Global L1 regularization coefficient.

`l2_reg_coef` Default: `0.0`. Global L2 regularization coefficient.

`init_weights_factor` Default: `1.0`. Global post-construction multiplicative scale applied to floating-point parameters.

`grad_clip_val` Default: `2.0`. Gradient clipping threshold.

`loss_l1_fraction` Default: `1.0`. Blend between L2 and L1 block losses; the default therefore selects pure L1 loss.

`loss_coef_observables` Default: `0.0`. Shared weight for band-energy and electron-count loss terms.

`loss_coef_forces` Default: `0.0`. Experimental band-energy derivative weight; not part of the validated objective.

`loss_coef_stress` Default: `0.0`. Reserved experimental band-energy cell-derivative weight; not part of the validated objective.

`use_lr_scheduler` Default: `True`. Boolean switch. When enabled, a reduce-on-plateau learning-rate scheduler is used.

`lr_scheduler_factor` Default: `0.5`. Positive float below one; sets the multiplicative decay factor.

`lr_scheduler_patience` Default: `60`. Nonnegative integer; sets plateau patience in epochs.

`lr_scheduler_min_lr` Default: `1e-8`. Nonnegative float; sets the minimum LR floor.

`lr_scheduler_target` Default: `val/hamiltonian_mae`. Logged metric name monitored by `ReduceLROnPlateau`.

`revert_on_spike` Default: `True`. Boolean switch for the revert-to-best checkpoint callback behavior.

`revert_monitor` Default: `val/hamiltonian_mae`. Metric name monitored by the revert-on-spike callback.

`revert_decay_patience` Default: `4`. Number of bad epochs before triggering revert or decay.

`revert_decay_rate` Default: `0.5`. Positive float below one; sets the LR decay applied after revert.

`revert_spike_factor` Default: `2.0`. Positive float that defines the spike threshold relative to the best score.

`checkpoint_monitor` Default: `val/hamiltonian_mae`. Metric used to select the best checkpoint.

Appendix A.7. Logging, Evaluation Artifacts, and Benchmarking

`run_name` Default: `mandala-run`. Human-readable run name.

`verbosity` Default: `1`. Integer verbosity level for summaries.

`wandb_project` Default: `None`. Weights & Biases project name.

`log_on_step` Default: `False`. Boolean switch for logging metrics on each training step.

`log_on_epoch` Default: `True`. Boolean switch for logging metrics on epoch aggregation.

`log_partial_gt_observables` Default: `True`. Boolean switch for logging hybrid observable diagnostics.

`log_per_irrep_metrics` Default: `True`. Boolean switch for computing and logging per-irrep metrics during training and evaluation.

`print_per_irrep_metrics` Default: `False`. Boolean switch for printing per-irrep metrics to the console during evaluation callbacks.

`log_per_irrep_images` Default: `False`. Boolean switch for saving per-irrep matrix comparison images in artifact callbacks.

`log_activation_mag` Default: `False`. Boolean switch for tracking and logging per-irrep activation magnitudes.

`log_data` Default: `False`. Boolean switch for detailed dataset-side logging.

`log_forward` Default: `False`. Boolean switch for detailed forward-pass logging.

`benchmark` Default: `False`. Boolean switch for timing and benchmark reporting paths.

`log_interval` Default: `10`. Positive integer epoch interval for detailed artifact logging.

`adaptive_log_interval` Default: `True`. Boolean switch for adaptive epoch logging cadence.

`video_max_atoms` Default: `6`. Positive integer or `None`; caps the atom count in matrix or video artifacts.

`log_model` Default: `False`. Boolean switch for logging the full model artifact to WandB.

Appendix A.8. Runtime, Device, Caching, and Reproducibility

`safety_checks` Default: `False`. Boolean switch for expensive validation checks on graph alignment, shape matching, and geometry consistency.

`dtype` Default: `torch.float32`. Default tensor dtype consulted by the network code.

`device` Default: `cuda`. Default device target as a torch device string.

`gpus` Default: `1`. Nonnegative integer GPU count request.

`num_workers` Default: `3`. Nonnegative integer or `None`; controls DataLoader worker count.

`save_dir` Default: `checkpoints`. Filesystem path root for checkpoints and artifacts.

`snapshot_cache_dir` Default: `None`. Filesystem path or `None`; location of raw snapshot cache and preprocessed sample cache.

`dataset_device` Default: `None`. Optional torch device string for processed dataset payloads.

`shuffle_snapshot_load_order` Default: `True`. Randomizes cache warm-up order without changing split membership.

`data_split_seed` Default: `42`. Seed defining train, validation, and test membership.

`seed` Default: `42`. Global random seed for reproducibility.

`tune` Default: `None`. Hyperparameter tuning backend selector such as `ray` or `wandb`.

Appendix A.9. Constraints and Appendix Notes

1. `train_on_energy=True` with `train_observables_on_gt=False` requires `matrix_targets` to include both `hamiltonian` and `density`.
2. `train_on_num_electrons=True` with `train_observables_on_gt=False` requires `matrix_targets` to include both `density` and `overlap`.
3. `train_on_energy=True` or `train_on_num_electrons=True` requires `loss_coef_observables != 0`.

4. `train_on_irrep_parts=True` cannot be combined with `train_target="irreps"`.
5. `partial_train="offdiag"` behaves differently depending on `separate_shifted_self`.
6. `hidden_irreps` overrides the automatic `build_hidden_irreps(l_max, hidden_base_dim, emb_use_odd_features)` logic completely.
7. `rescale_density_to_num_electrons=True` affects artifact and metric evaluation.

References

[1] P. Hohenberg, W. Kohn, Inhomogeneous electron gas, Phys. Rev. 136 (1964) B864–B871. doi:10.1103/PhysRev.136.B864.

[2] W. Kohn, L. J. Sham, Self-Consistent Equations Including Exchange and Correlation Effects, Phys. Rev. 140 (1965) A1133–A1138. doi:10.1103/PhysRev.140.A1133.

[3] L. Fiedler, K. Shah, M. Bussmann, A. Cangi, Deep dive into machine learning density functional theory for materials science and chemistry, Phys. Rev. Mater. 6 (2022) 040301. doi:10.1103/PhysRevMaterials.6.040301.
URL https://link.aps.org/doi/10.1103/PhysRevMaterials.6.040301

[4] J. A. Ellis, L. Fiedler, G. A. Popoola, N. A. Modine, J. A. Stephens, A. P. Thompson, A. Cangi, S. Rajamanickam, Accelerating finite-temperature Kohn-Sham density functional theory with deep neural networks, Phys. Rev. B 104 (2021) 035120. doi:10.1103/PhysRevB.104.035120.
URL https://link.aps.org/doi/10.1103/PhysRevB.104.035120

[5] L. Fiedler, N. A. Modine, K. D. Miller, A. Cangi, Machine learning the electronic structure of matter across temperatures, Phys. Rev. B 108 (2023) 125146. doi:10.1103/PhysRevB.108.125146.
URL https://link.aps.org/doi/10.1103/PhysRevB.108.125146

[6] L. Fiedler, N. A. Modine, S. Schmerler, D. J. Vogel, G. A. Popoola, A. P. Thompson, S. Rajamanickam, A. Cangi, Predicting electronic structures at any length scale with machine learning, npj Comput Mater 9 (1)

(2023) 1–10. doi:10.1038/s41524-023-01070-z.
URL https://www.nature.com/articles/s41524-023-01070-z

[7] L. Fiedler, Z. A. Moldabekov, X. Shao, K. Jiang, T. Dornheim, M. Pavanello, A. Cangi, Accelerating equilibration in first-principles molecular dynamics with orbital-free density functional theory, Phys. Rev. Res. 4 (2022) 043033. doi:10.1103/PhysRevResearch.4.043033.
URL https://link.aps.org/doi/10.1103/PhysRevResearch.4.043033

[8] S. A. Ghasemi, T. D. Kühne, Artificial neural networks for the kinetic energy functional of non-interacting fermions, J. Chem. Phys. 154 (7) (2021) 074107. doi:10.1063/5.0037319.
URL https://doi.org/10.1063/5.0037319

[9] A. Chandrasekaran, D. Kamal, R. Batra, C. Kim, L. Chen, R. Ramprasad, Solving the electronic structure problem with machine learning, npj computational materials 5 (1) (2019) 22. doi:10.1038/s41524-019-0162-7.

[10] M. Tsubaki, T. Mizoguchi, Quantum Deep Field: Data-Driven Wave Function, Electron Density Generation, and Atomization Energy Prediction and Extrapolation with Machine Learning, Phys. Rev. Lett. 125 (2020) 206401. doi:10.1103/PhysRevLett.125.206401.
URL https://link.aps.org/doi/10.1103/PhysRevLett.125.206401

[11] X. Shao, L. Paetow, M. E. Tuckerman, M. Pavanello, Machine learning electronic structure methods based on the one-electron reduced density matrix, Nature Communications 14 (1) (Oct. 2023). doi:10.1038/s41467-023-41953-9.
URL http://dx.doi.org/10.1038/s41467-023-41953-9

[12] F. Brockherde, L. Vogt, L. Li, M. E. Tuckerman, K. Burke, K.-R. Müller, Bypassing the Kohn-Sham equations with machine learning, Nat. Commun. 8 (1) (2017) 872. doi:10.1038/s41467-017-00839-3.

[13] E. Kocer, T. W. Ko, J. Behler, Neural network potentials: A concise overview of methods, Annual Review of Physical Chemistry 73 (Volume 73, 2022) (2022) 163–186. doi:https:

//doi.org/10.1146/annurev-physchem-082720-034254.
URL https://www.annualreviews.org/content/journals/10.1146/annurev-physchem-082720-034254

[14] J. A. Rackers, L. Tecot, M. Geiger, T. E. Smidt, A recipe for cracking the quantum scaling limit with machine learned electron densities, Machine Learning: Science and Technology 4 (1) (2023) 015027. doi:10.1088/2632-2153/acb314.
URL https://dx.doi.org/10.1088/2632-2153/acb314

[15] H. Li, Z. Wang, N. Zou, M. Ye, R. Xu, X. Gong, W. Duan, Y. Xu, Deep-learning density functional theory hamiltonian for efficient ab initio electronic-structure calculation, Nat. Comput. Sci. 2 (6) (2022) 367–377. doi:10.1038/s43588-022-00265-6.
URL https://doi.org/10.1038/s43588-022-00265-6

[16] H. Li, Z. Tang, X. Gong, N. Zou, W. Duan, Y. Xu, Deep-learning electronic-structure calculation of magnetic superstructures, Nat. Comput. Sci. 3 (4) (2023) 321–327. doi:10.1038/s43588-023-00424-3.
URL https://doi.org/10.1038/s43588-023-00424-3

[17] X. Gong, H. Li, N. Zou, R. Xu, W. Duan, Y. Xu, General framework for e(3)-equivariant neural network representation of density functional theory hamiltonian, Nat. Commun. 14 (1) (2023) 2848. doi:10.1038/s41467-023-38468-8.
URL https://doi.org/10.1038/s41467-023-38468-8

[18] Z. Tang, H. Li, P. Lin, X. Gong, G. Jin, L. He, H. Jiang, X. Ren, W. Duan, Y. Xu, A deep equivariant neural network approach for efficient hybrid density functional calculations, Nat. Commun. 15 (1) (2024) 8815. doi:10.1038/s41467-024-53028-4.
URL https://doi.org/10.1038/s41467-024-53028-4

[19] X. Gong, S. G. Louie, W. Duan, Y. Xu, Generalizing deep learning electronic structure calculation to the plane-wave basis, Nat. Comput. Sci. 4 (10) (2024) 752–760. doi:10.1038/s43588-024-00701-9.
URL https://doi.org/10.1038/s43588-024-00701-9

[20] Z. Tang, H. Chen, Y. Li, Y. Qian, Y. Wang, W. Fu, J. Li, C. Si, W. Duan, J. Chen, Y. Xu, Deep-learning electronic structure calculations, Nat. Comput. Sci. 5 (12) (2025) 1133–1146. doi:10.1038/

s43588-025-00932-4.
URL https://doi.org/10.1038/s43588-025-00932-4

[21] M. Born, R. Oppenheimer, Zur Quantentheorie der Molekeln, Ann. Phys. 389 (20) (1927) 457–484. doi:10.1002/andp.19273892002.

[22] T. Ozaki, Variationally optimized atomic orbitals for large-scale electronic structures, Phys. Rev. B 67 (15) (2003) 155108. doi:10.1103/PhysRevB.67.155108.
URL https://doi.org/10.1103/PhysRevB.67.155108

[23] T. Ozaki, H. Kino, Numerical atomic basis orbitals from h to kr, Phys. Rev. B 69 (19) (2004) 195113. doi:10.1103/PhysRevB.69.195113.
URL https://doi.org/10.1103/PhysRevB.69.195113

[24] V. Blum, R. Gehrke, F. Hanke, P. Havu, V. Havu, X. Ren, K. Reuter, M. Scheffler, Ab initio molecular simulations with numeric atom-centered orbitals, Comput. Phys. Commun. 180 (11) (2009) 2175–2196. doi:10.1016/j.cpc.2009.06.022.
URL https://doi.org/10.1016/j.cpc.2009.06.022

[25] Q. Sun, X. Zhang, S. Banerjee, P. Bao, M. Barbry, N. S. Blunt, N. A. Bogdanov, G. H. Booth, J. Chen, Z.-H. Cui, J. J. Eriksen, Y. Gao, S. Guo, J. Hermann, M. R. Hermes, K. Koh, P. Koval, S. Lehtola, Z. Li, J. Liu, N. Mardirossian, J. D. McClain, M. Motta, B. Mussard, H. Q. Pham, A. Pulkin, W. Purwanto, P. J. Robinson, E. Ronca, E. R. Sayfutyarova, M. Scheurer, H. F. Schurkus, J. E. T. Smith, C. Sun, S.-N. Sun, S. Upadhyay, L. K. Wagner, X. Wang, A. White, J. D. Whitfield, M. J. Williamson, S. Wouters, J. Yang, J. M. Yu, T. Zhu, T. C. Berkelbach, S. Sharma, A. Y. Sokolov, G. K.-L. Chan, Recent developments in the PySCF program package, J. Chem. Phys. 153 (2) (2020) 024109. doi:10.1063/5.0006074.
URL https://doi.org/10.1063/5.0006074

[26] L. Fiedler, N. Hoffmann, P. Mohammed, G. A. Popoola, T. Yovell, V. Oles, J. A. Ellis, S. Rajamanickam, A. Cangi, Training-free hyperparameter optimization of neural networks for electronic structures in matter, Mach. Learn.: Sci. Technol. 3 (4) (2022) 045008. doi:10.1088/2632-2153/ac9956.